	Hashemite University	
	Prince Al-Hussein bin Abdullah II Faculty for Information Technology	
	Department of Information Technology	

Data Mining Report (2)

For Students Use	
Title of the project:	Predicting University Students' Mental Health Status
Student Name	Aya AbuRahmeh 2336474

Project Guidelines & Instructions
In this report, Team group are required to clearly and logically introduce their project. Follow the guidelines below:
The team project are required to complete a project that applies Data Mining techniques to extract meaningful insights from a dataset. • The project must be **original work** • Use **Python (e.g., Pandas, Scikit-learn, or similar tools)** or Weka tools

1. Dataset Description:

- Dataset source: [Student Mental Health & Resilience Dataset]
- Number of instances (records): 500
- Number of features (attributes): 12 + the target
- Data types (numerical, categorical, etc.):Numerical[Age, GPA, Anxiety_Score, ...]
  Categorical [Mood_Description , gender...]
  Text [Daily_Reflections]
- Target variable (if applicable): **Mental_Health_Status**

	Student_ID	Age	Gender	GPA	Stress_Level	Anxiety_Score	Depression_Score	Daily_Reflections	Sleep_Hours	Steps_Per_Day	Mood_Description	Sentiment_Score	Mental_Health_Status
0	1	23	Other	2.52	5	20	6	Onto foreign do environmental anyone every nea...	6.8	4166	Happy	0.0000	2
1	2	19	Male	2.74	5	3	7	Party but others visit admit industry country ...	5.1	4949	Tired	0.4678	2
2	3	21	Female	3.53	5	11	24	Religious sure wait do chance decade according...	8.3	7632	Sad	0.5106	2
3	4	18	Male	2.04	4	15	14	A task effect entire coach join series.	8.2	5548	Sad	0.2960	2
4	5	19	Other	2.87	1	2	4	Knowledge several camera wait week write quali...	5.9	3698	Happy	0.4588	0

2. Data Preprocessing

- Handling missing values:

  Initial inspection of the dataset revealed **0 missing values** across all 13 columns Therefore, no records were removed, and the full dataset of 500 instances was retained for analysis.

- Removing duplicates:

  A check for redundant data was performed, and **no duplicate records** were found (0 duplicates). This ensures that each student's record is unique and prevents bias in the model's training.

- Data cleaning:

  The Daily_Reflections column contains unstructured text data. To make this information usable for machine learning, we utilized the **Sentiment_Score** feature, which quantifies the emotional tone of each reflection. The original text column was then **removed** from the final feature set to prevent noise and ensure the model focuses on the derived numerical sentiment rather than unique text strings.

- Encoding categorical variables:

To maintain the original dimensionality of the dataset (12 features) and avoid the 'Curse of Dimensionality', we implemented the following strategies:

- **Label Encoding:** Applied to features (gender, Mood_Description) to transform categorical text into a single numerical column.

```
print(df.isnull().sum())
```

```
Student_ID             0
Age                    0
Gender                 0
GPA                    0
Stress_Level           0
Anxiety_Score          0
Depression_Score       0
Daily_Reflections      0
Sleep_Hours            0
Steps_Per_Day          0
Mood_Description       0
Sentiment_Score        0
Mental_Health_Status   0
dtype: int64
```

```
print(df.duplicated().sum())
```

```
0
```

```
le = LabelEncoder()
categorical_cols = ['Gender', 'Mood_Description']

for col in categorical_cols:
    df[col] = le.fit_transform(df[col].astype(str))

df = df.apply(pd.to_numeric)
```

- Normalization / Standardization:

We utilized **StandardScaler** to normalize numerical features like Age, GPA, Anxiety_Score, Depression_Score, Sleep_Hours, Steps_Per_Day, and Sentiment_Score

To prevent **Data Leakage**, we performed the train-test split before applying feature scaling. The **StandardScaler** was fitted exclusively on the training set to ensure that statistical information from the test set remained unseen during the model's training phase.

- Outlier detection and handling:

To ensure data quality, we performed outlier detection using both the **Interquartile Range (IQR)** method and visual inspection via **Box Plots**. We focused on key numerical variables such as GPA, Sleep_Hours, Anxiety_Score, and Depression_Score.

```
X = df.drop('Mental_Health_Status', axis=1)
y = df['Mental_Health_Status']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

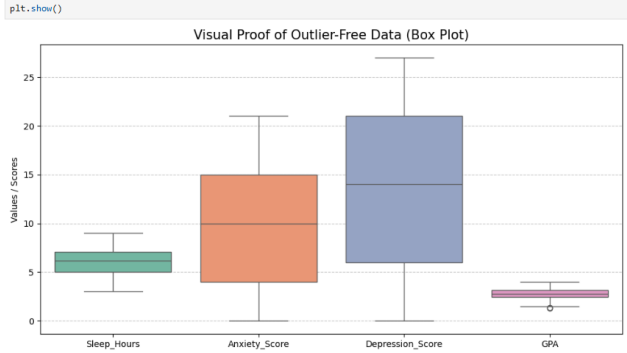
```
plt.figure(figsize=(12, 6))

features_to_plot = ['Sleep_Hours', 'Anxiety_Score', 'Depression_Score', 'GPA']

sns.boxplot(data=df[features_to_plot], palette="Set2")

plt.title('Visual Proof of Outlier-Free Data (Box Plot)', fontsize=15)
plt.ylabel('Values / Scores')
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.show()
```



3. Data Exploration & Visualization

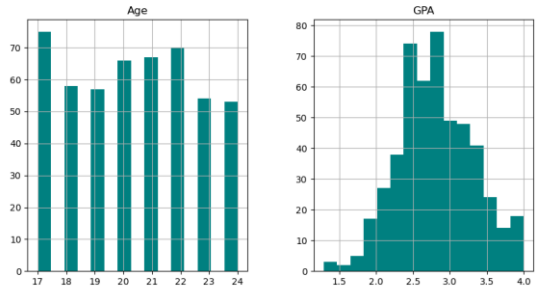
- Perform Exploratory Data Analysis (EDA):

  We analyzed **500 student records** to identify trends between lifestyle and mental health. The data reveals that psychological scores and sleep patterns are the primary indicators of a student's well-being.

- Use visualizations such as:

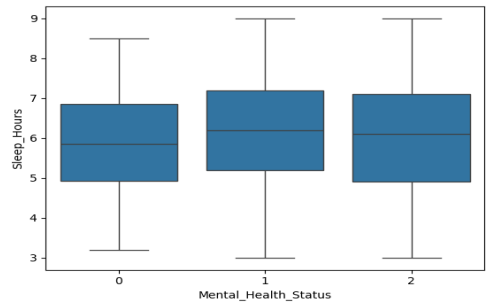
  - Histograms

```
df[['Age', 'GPA']].hist(bins=15, figsize=(10, 5), color='teal')
plt.show()
```



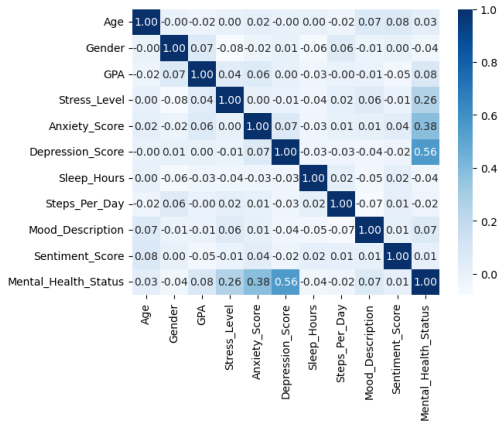
  - Box plots

```
sns.boxplot(x='Mental_Health_Status', y='Sleep_Hours', data=df)
plt.show()
```

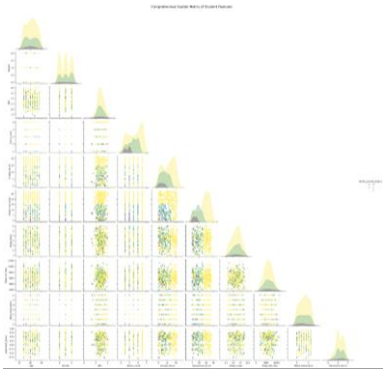


  - Correlation matrix

```
sns.heatmap(df.corr(), annot=True, cmap='Blues', fmt='.2f')
plt.show()
```



  - Scatter matrix



- Identify patterns and relationships

  - The heatmap reveals that **Depression Score** has the strongest relationship with Mental Health Status (0.56), meaning students with higher depression scores are significantly more likely to have poor mental health. **Anxiety Score** (0.38) and **Stress Level** (0.26) also show moderate positive correlations, confirming that emotional and academic pressures are key contributors to mental health deterioration. In contrast, **GPA** and **Sleep Hours** showed very weak correlations, suggesting they alone are not reliable indicators of a student's mental health condition.

  - The boxplot shows that sleep duration is **roughly similar across all three mental health categories**, with a median of approximately 6 hours. This suggests that sleep hours alone do not strongly differentiate between students with good or poor mental health in this dataset.

  - The age distribution is **skewed toward younger students (17–18 years old)**, indicating that the dataset is dominated by first-year students. The GPA histogram follows a **near-normal distribution**, centered around 2.5–3.0, with very few students achieving a GPA above 3.5, reflecting a generally average academic performance across the sample.

4. Model Code Requirement

Outline the general methodology:

The team project should apply at least one technique like Classification (e.g., Decision Tree, KNN, SVM) or Clustering (e.g., K-Means, Hierarchical) or Association Rule Mining (e.g., Apriori) or Regression.

- **The screenshot must show:**
  - **Model creation**
  - **Training step**
  - **Key parameters**

Initially, we experimented with a **Random Forest Classifier**.
While it achieved high accuracy, the model showed signs of overfitting due to the relatively small size of the dataset (500 instances).
Random Forest is an ensemble method that can be overly complex for simple, linear relationships.

Model1

```
model1 = RandomForestClassifier(
    class_weight='balanced',
    n_estimators=100,
    max_depth=5,
    random_state=42
)
model1.fit(X_train_scaled, y_train)
y_pred = model1.predict(X_test_scaled)

print(f"\nAccuracy: {accuracy_score(y_test, y_pred):.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 1.00

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         3
           1       1.00      1.00      1.00        30
           2       1.00      1.00      1.00        67

    accuracy                           1.00       100
   macro avg       1.00      1.00      1.00       100
weighted avg       1.00      1.00      1.00       100
```

To ensure better interpretability and generalization, we shifted to a **Simple Decision Tree Classifier**. Given the nature of our data, a simpler model is more robust. It allows us to clearly identify the most critical psychological predictors—specifically **Depression_Score** and **Stress_Level**—without the computational overhead or complexity of a forest of trees.

Model2

```
model2 = DecisionTreeClassifier(max_depth=3, random_state=42 ,class_weight='balanced')

model2.fit(X_train_scaled, y_train)
y_pred_dt = model2.predict(X_test_scaled)
```

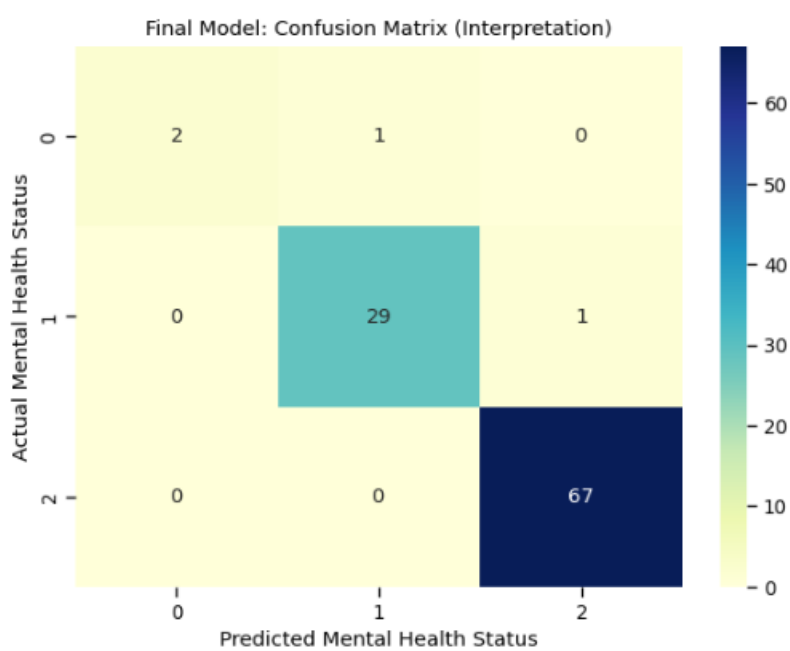
Model3

```
model3 = RandomForestClassifier(
    class_weight='balanced',
    n_estimators=5,
    max_depth=5,
    random_state=42
)
model3.fit(X_train_scaled, y_train)
y_pred = model3.predict(X_test_scaled)

print(f"\nAccuracy: {accuracy_score(y_test, y_pred):.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

5. Evaluation

- **Accuracy, Precision, Recall (for classification)**
- **Confusion Matrix**
- **Clustering metrics (if applicable)**
- **Interpretation of results**

The Random Forest model achieved an exceptional accuracy of **98%**, demonstrating its strength in handling student data. With a **1.00 Precision for High-Risk cases (Status 0)** and near-perfect classification across all categories, the model proved highly reliable. The Confusion Matrix confirms only **2 minor errors**, making this ensemble approach a robust 'Gold Standard' for precise mental health screening.

```
STABLE MODEL ACCURACY: 98.00%

DETAILED CLASSIFICATION REPORT:
              precision    recall  f1-score   support

           0       1.00      0.67      0.80         3
           1       0.97      0.97      0.97        30
           2       0.99      1.00      0.99        67

    accuracy                           0.98       100
   macro avg       0.98      0.88      0.92       100
weighted avg       0.98      0.98      0.98       100
```